# CSCI 350:
# Fundamentals of Operating Systems

## Synchronization

***Prof. Tatyana Ryutov***

This content is protected and may not be shared, uploaded, or distributed.

# Outline

- Review
- Synchronization inro
- Synchronization constructs
  - Locks, lock implementation
  - Condition variables
  - Semaphores

©Tatyana Ryutov

# Reminder

- Midterm1: February 20th official time1:00-3:00pm
- Your exam must be submitted by midnight on February 20th
- The exam will be conducted remotely
- No lecture on this day
- Exam covers lectures 1 – 5 – all topics discussed in these lectures
- You may post your exam-related questions during the official exam **time (1-3pm)** via Piazza. Use “private question” setting
- We will not monitor Piazza for your exam-related questions after 2pm
- List of topics posted on Piazza
- Practice exam will be posted on the Brightspace->Quizzes (not graded)

©Tatyana Ryutov

# Exam Policies

- Access the exam via the Blackboard. It will be available in Assignments on the specified day and time.
- Exam is closed book/notes.
- You'll have 2hrs + 15-min grace period to complete the exam.

- You are allowed one 8.5" x 11" handwritten note sheet (both sides). NO TYPED NOTES ARE ALLOWED!
- You may post your exam-related questions during the exam via Piazza. Use "private question" setting.
- After 135 minutes, your exam will be automatically submitted. You will not be able to pause the exam.
- One restroom break is allowed (5 minutes max).
- During the exam you'll record yourself: **share your screen** where you'll be inputting the answers, microphone and camera must be on. We should be able to see your face.
- If you experience a problem with the recording, let us know immediately via Piazza!
- You can copy your answers to a text editor and save them as a precaution.
- **Only the Blackboard, Piazza, and text editor should be opened. All other browser tabs must be closed.**
- **No second screens are allowed.**
- You need to **show us** your cheat sheet (both sides) before taking the exam.
- After the exam is over and your recording is ready, submit the link via a Google form on Piazza.
- **We will not grade your exam if you do not provide your recording, or if the recording does not satisfy the above requirements.**
- **If you have never recorded your Zoom meeting, practice well before the exam!**
- To schedule the recording login into Zoom using your USC credentials, record to the cloud.
- In order to not forget to record yourself, select "record the meeting automatically" when scheduling your meeting.

©Tatyana Ryutov

# Your Specific Questions

- How much of Project 1 compared to lecture material would be in the exam? Which should be focused on more for studying?
- Why do we need to fork() processes/why is duplicating necessary?
- Do we have to use fork()  at all?
- What is the main difference between thread and process communication?
- Why would we choose to use threads instead of processes in certain applications?
- What happens internally if we lose a process's state during a switch?
- Difference between multithreading, multiprogramming, and multiprocessing

©Tatyana Ryutov

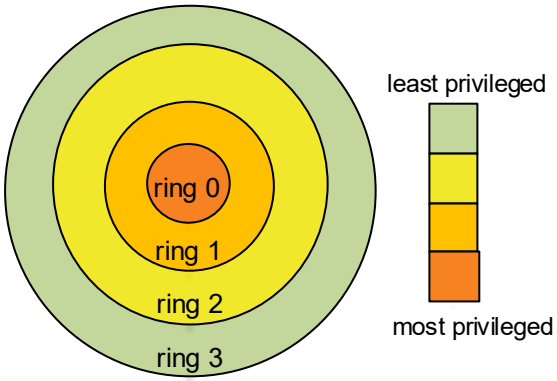
# Your Review Topics

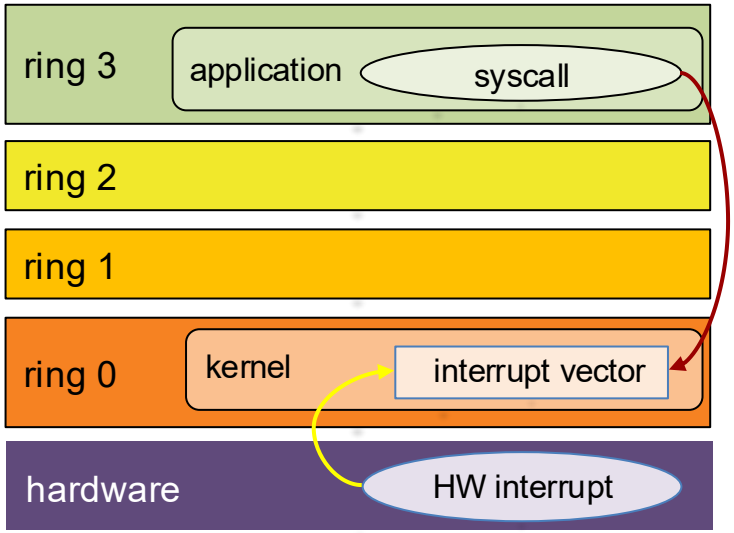
- Interrupts vs. traps
- Kernel/user stacks
- System call processing
- Unix process management syscalls
- Fork() and output of calling fork()
- Context switch vs. process switch
- Process vs. thread
- Process vs. thread switch
- User-level threads vs. kernel-level threads

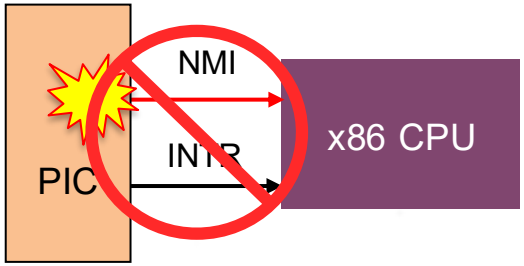
©Tatyana Ryutov

# Review: Interrupts vs. Traps

- On interrupt:
  1. HW interrupts CPU execution
  2. CPU switches to interrupt vector
  3. CPU switches to kernel mode and jumps to interrupt handler in OS
- On trap:
  1. CPU switches to interrupt vector
  2. CPU switches to kernel mode and execute OS instructions (syscall handler)



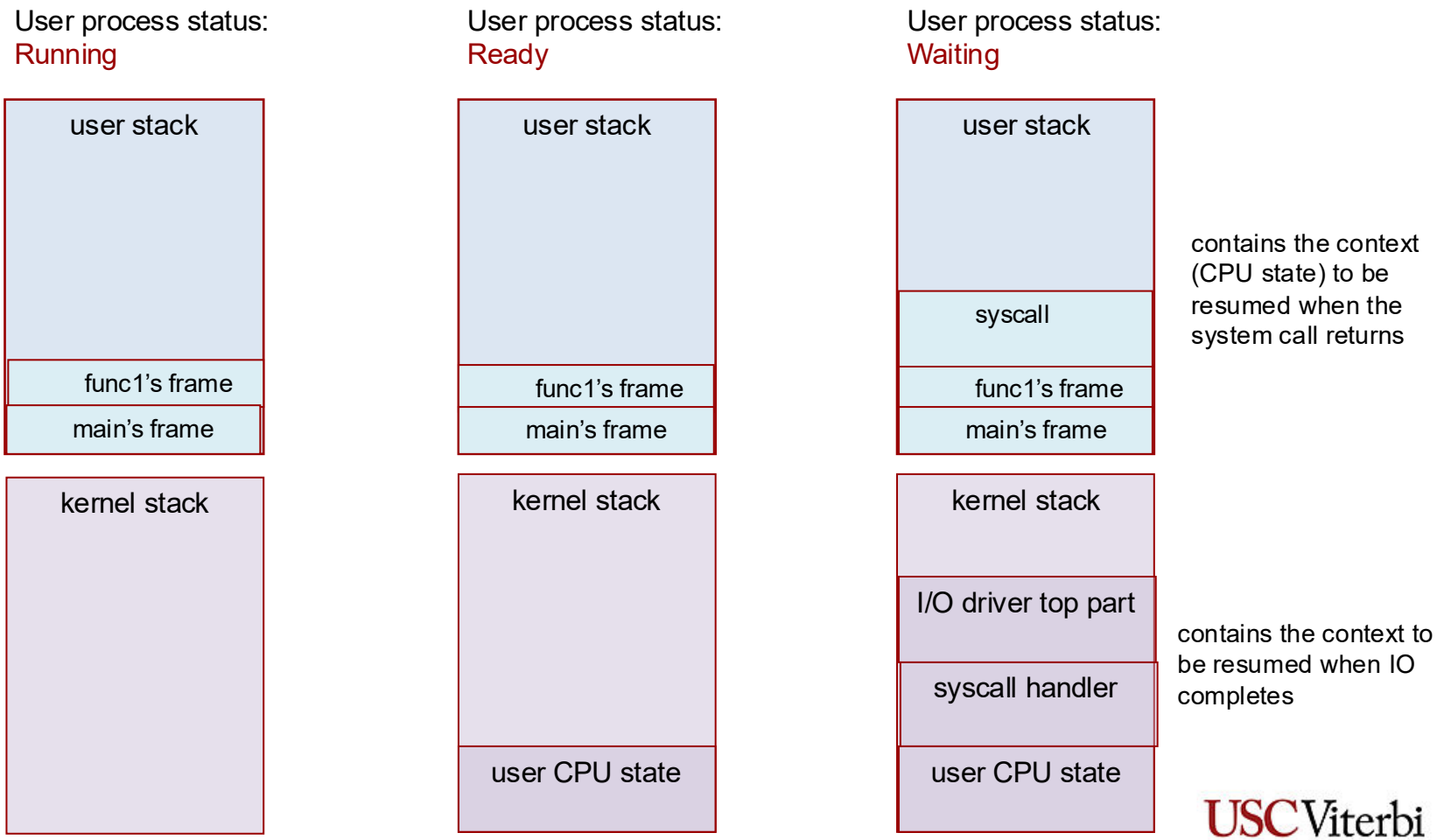




©Tatyana Ryutov

PIC programmable interrupt controller

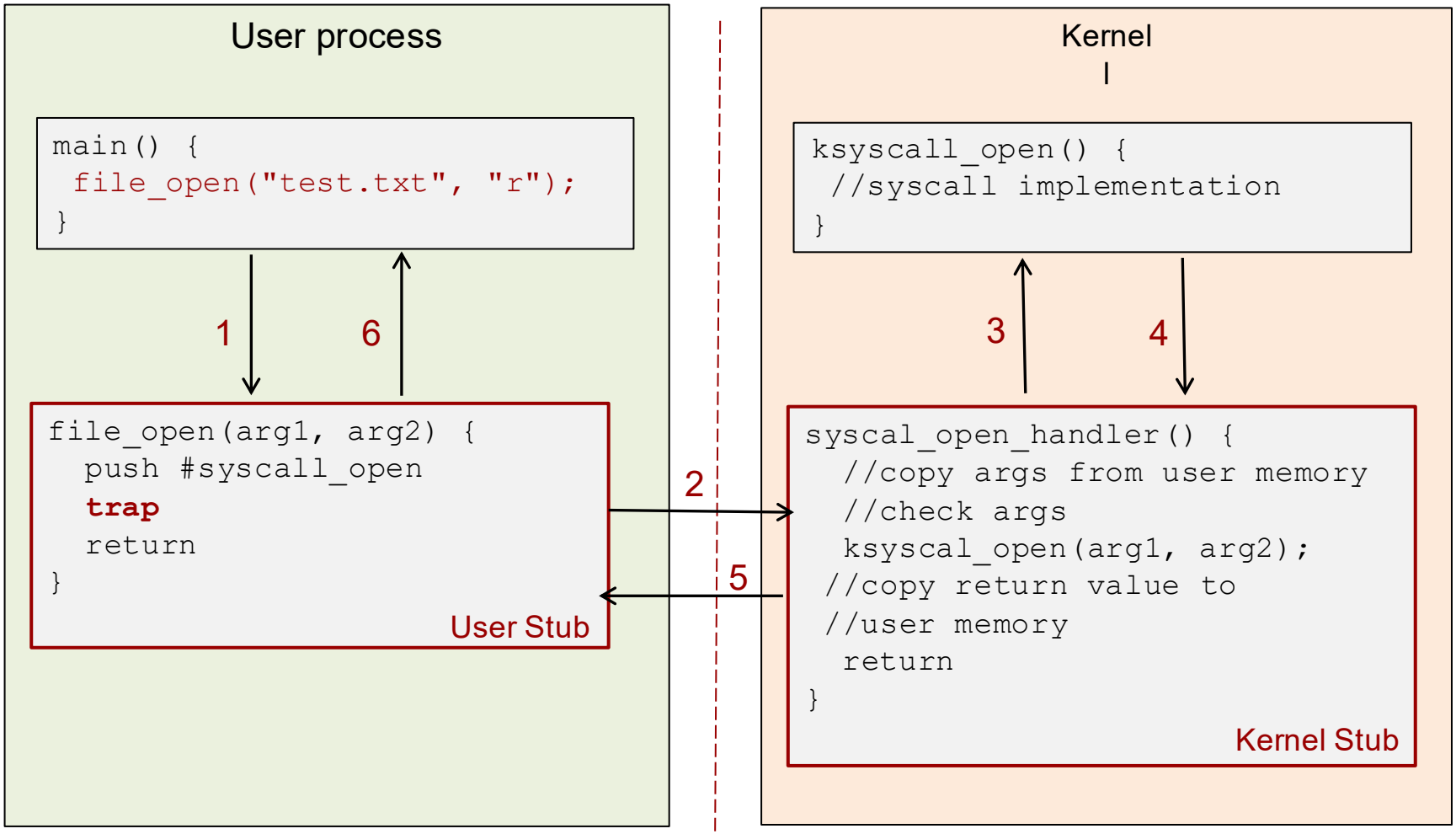
# Review: When syscall occurs, what stack is used?

- On a syscall, the processor switches to the kernel stack, jumps to kernel mode, and starts executing the kernel instructions that implement the syscall




©Tatyana Ryutov

# Review: Syscall Processing Overview




User-mode execution
(code from program ELF)

Kernel-mode execution
(code from kernel binary)

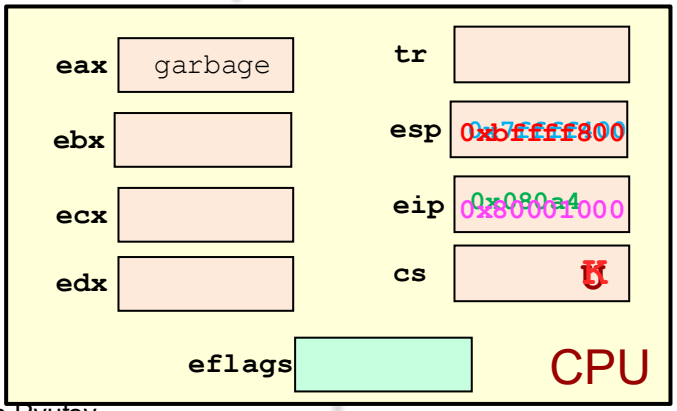
©Tatyana Ryutov

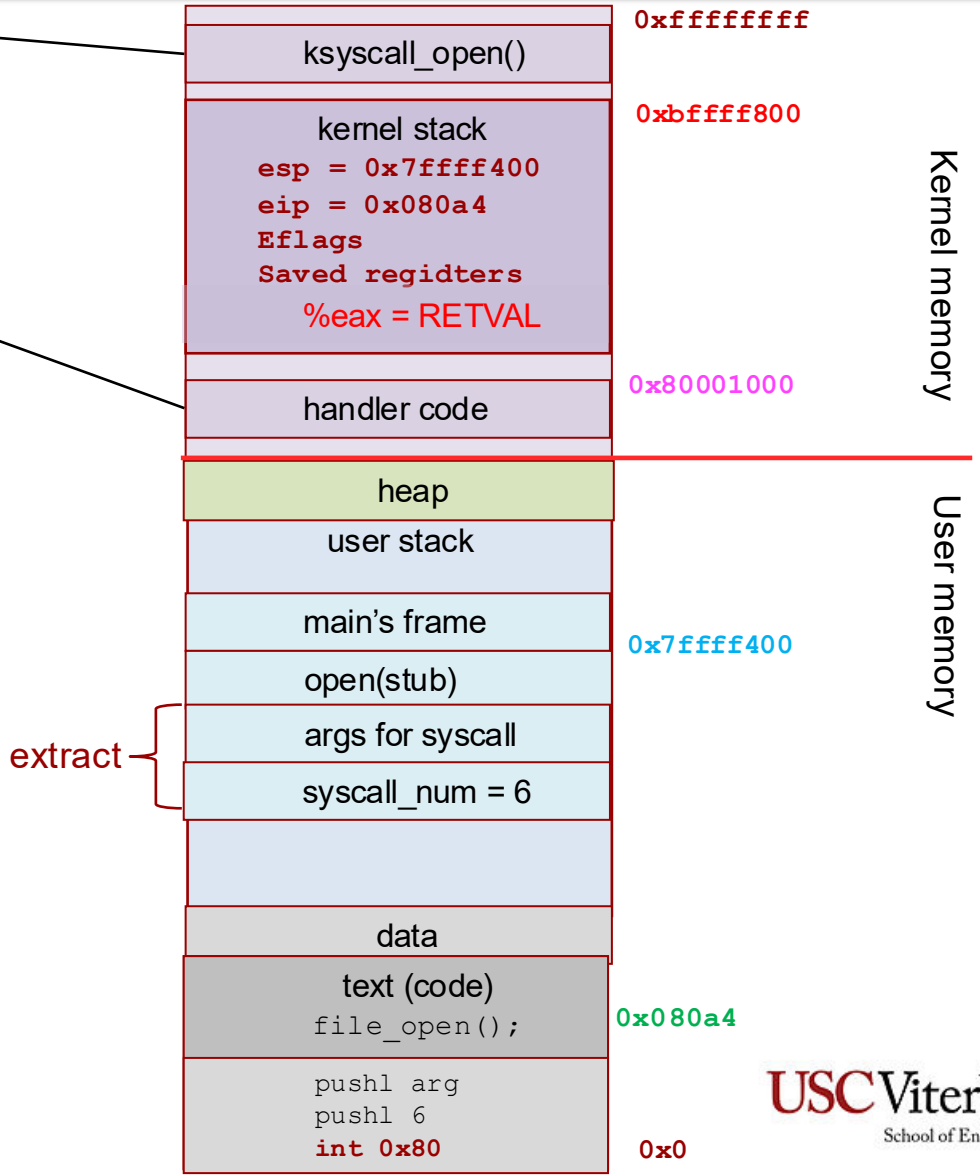
USC Viterbi School of Engineering

# Review: Syscall Execution Example

```
int ksyscall_open(…)
{
  //Extract arguments
  //Perform computation
  return value;
}
```

```
HANDLER30:
{
  pushad
  //Extract syscall num
  //If num == 0 …
  //If num = 1 …
  //...
  //If num == 6
  call ksyscall_open
  //Store rectal in %eax
   ...
   popad
  iret
}
```



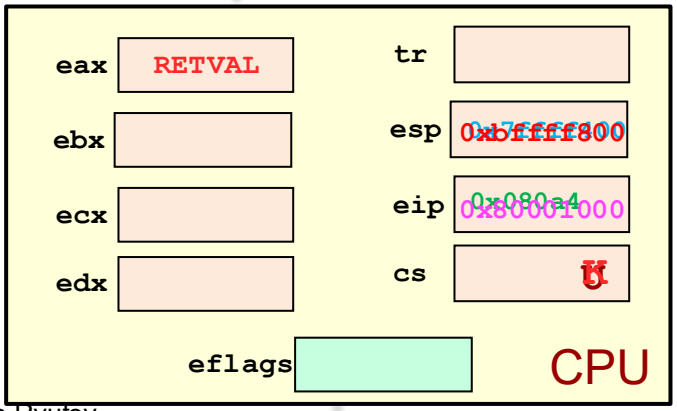

©Tatyana Ryutov

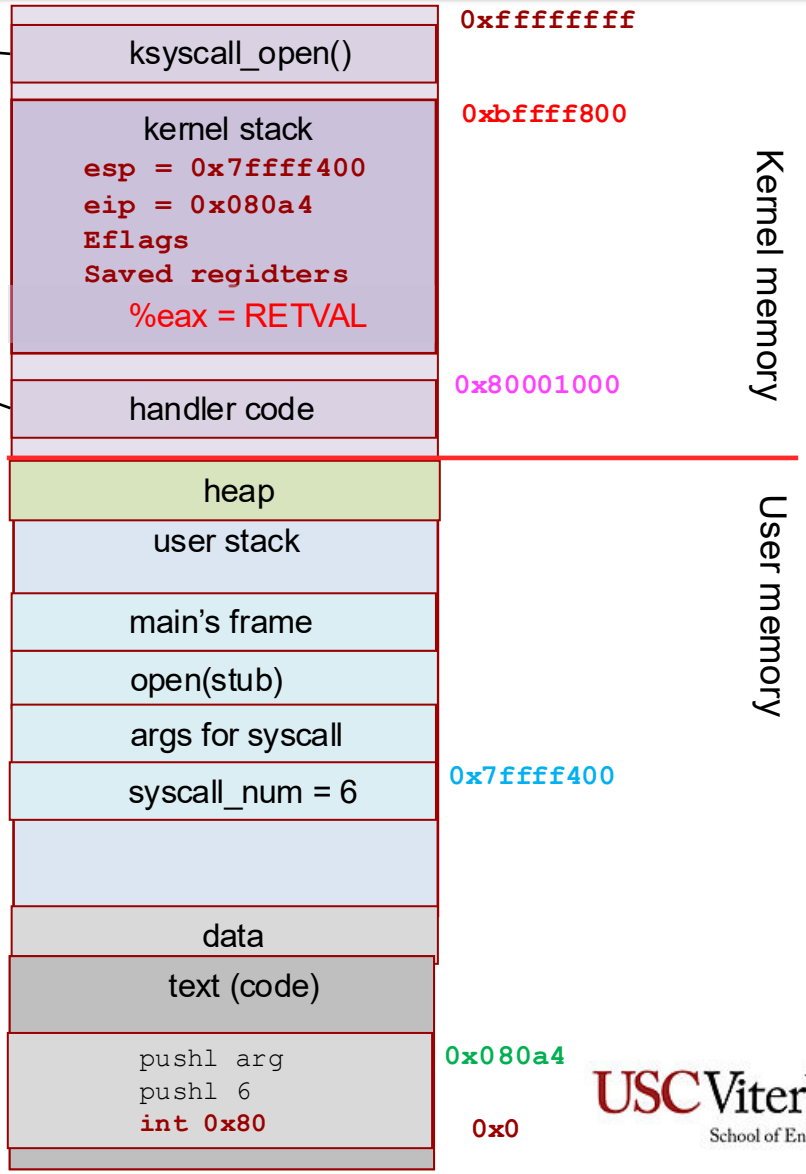
USC Viterbi School of Engineering

# Review: Syscall Execution Example

```
int ksyscall_open(…)
{
  //Extract arguments
  //Perform computation
  return value;
}
```

```
HANDLER30:
{
  pushad
  //Extract syscall num
  //If num == 0 …
  //If num = 1 …
  //...
  //If num == 6
  call ksyscall_open
  //Store rectal in %eax
   ...
   popad
  iret
}
```

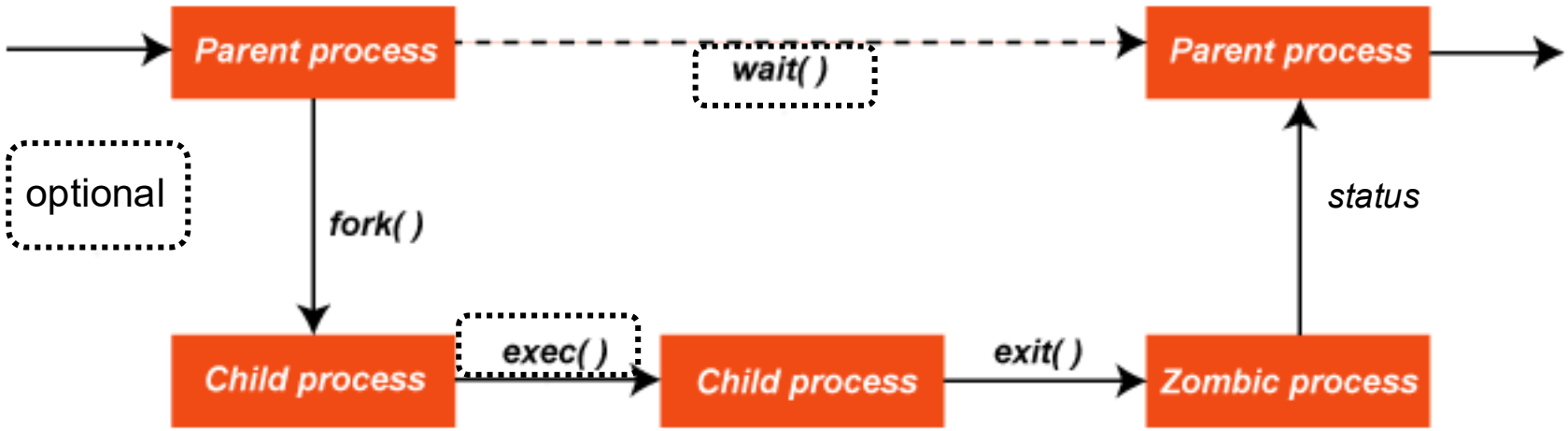



©Tatyana Ryutov

USC Viterbi School of Engineering

# Review: Unix Process Management




```
int pid = fork();
```
Create a new process that is a clone of its parent

```
exec*("program" [,argvp, envp]);
```
Overlay the calling process virtual memory with a new program, and transfer control to it

```
exit(status);
```
Exit with status, destroying the process: all of the memory and resources associated with it are deallocated. However, the process's entry in the process table remains

```
int pid = wait*(&status);
```
Wait for exit (or other status change) of a child

©Tatyana Ryutov

USC Viterbi School of Engineering

# Understanding `fork()`

Process X (parent)

pid = X

```
pid_t pid = fork();
if (pid == 0) {
   printf("hello from child\n");
} else {
   printf("hello from parent\n");
}
```

fork()

Process Y (child)

pid = Y

```
pid_t pid = fork();
if (pid == 0) {
   printf("hello from child\n");
} else {
   printf("hello from parent\n");
}
```

```
pid_t pid = fork();
if (pid == 0) {
   printf("hello from child\n");
} else {
   printf("hello from parent\n");
}
```

pid = Y

```
pid_t pid = fork();
if (pid == 0) {
   printf("hello from child\n");
} else {
   printf("hello from parent\n");
}
```

pid = 0

```
pid_t pid = fork();
if (pid == 0) {
   printf("hello from child\n");
} else {
   printf("hello from parent\n");
}
```

```
pid_t pid = fork();
if (pid == 0) {
   printf("hello from child\n");
} else {
   printf("hello from parent\n");
}
```

**hello from parent**

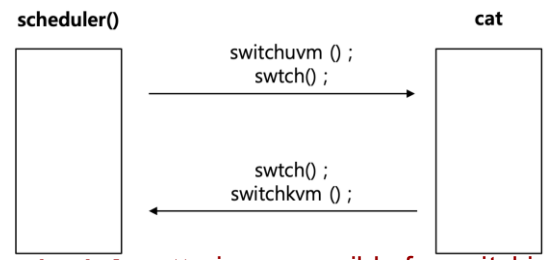
**hello from child**

©Tatyana Ryutov

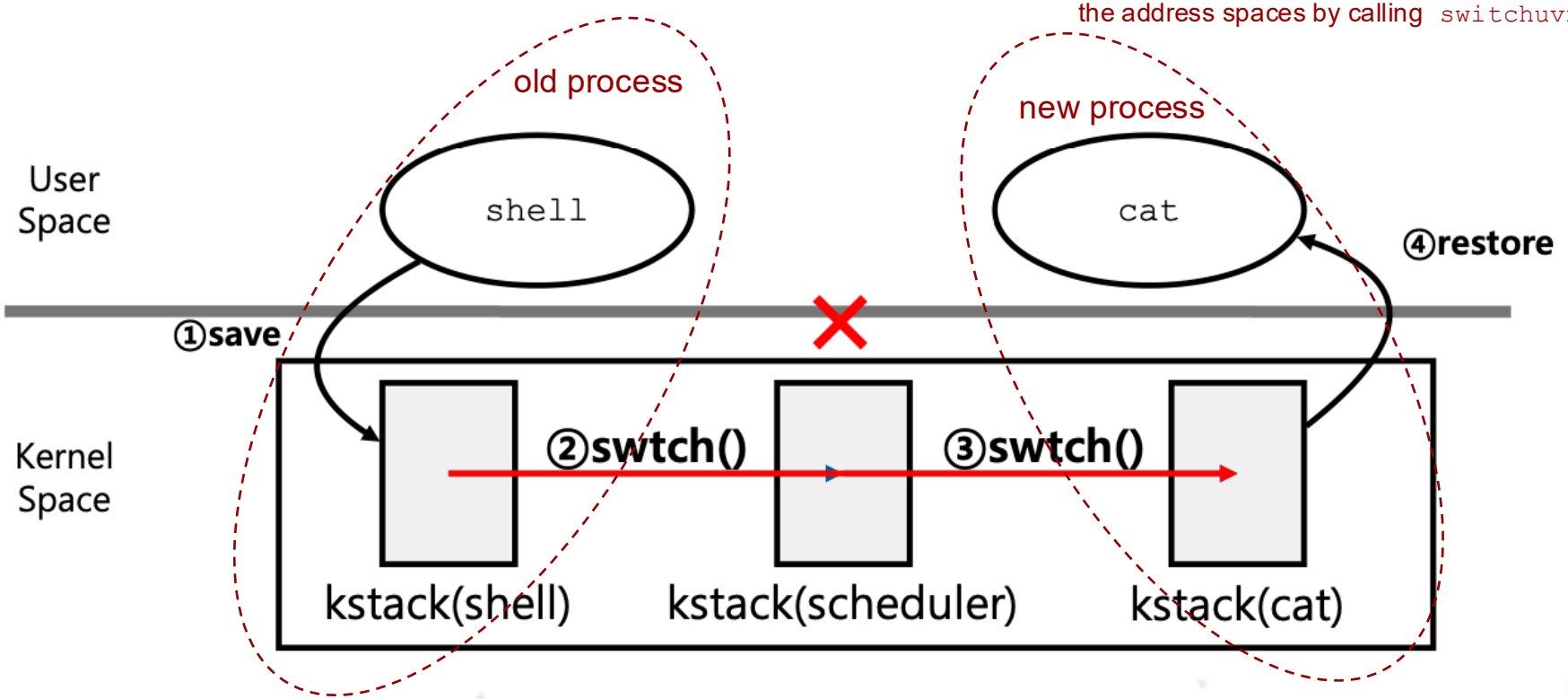
USC Viterbi School of Engineering

# Review: Process Switch and Context Switch

1. Stack switch from user to kernel
2. Context switch to the local CPU's scheduler thread
3. Context switch to a new process's kernel thread
4. Stack switch from kernel to user




`scheduler()` is responsible for switching the address spaces by calling `switchuvm`




©Tatyana Ryutov

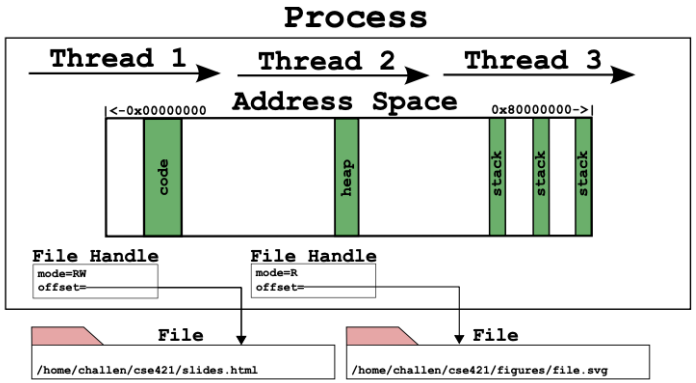
# Process/Context/Mode Switch Key Differences

- **Purpose**:
  - Context Switch enables CPU to handle multiple tasks
  - Process Switch enables CPU to handle multiple processes by switching between them
  - Mode Switch allows the CPU to change privilege level to perform of tasks that require privileged instructions
- **Changes:**
  - Context Switch: the task that the CPU is working on changes
  - Process Switch: the process that the CPU is working on changes
  - Mode Switch: the operating mode of the CPU changes
- **Overhead:**
  - Context Switch: generally has less overhead because the CPU needs to save limited state
  - Process Switch: generally has more overhead that context switch because the CPU needs to save and load entire process states
  - Mode Switch: typically has less overhead since it's just changing the CPU's privilege mode

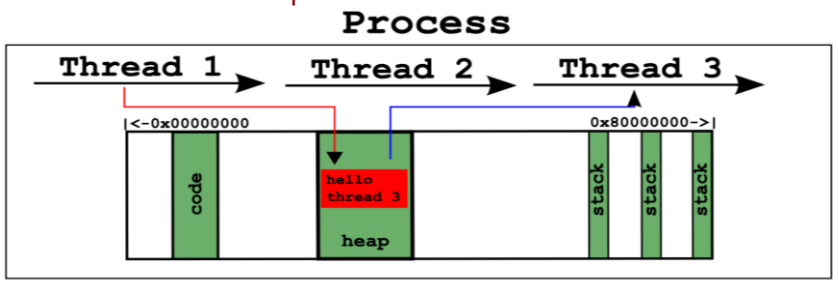
©Tatyana Ryutov

# Review: Process vs. Thread

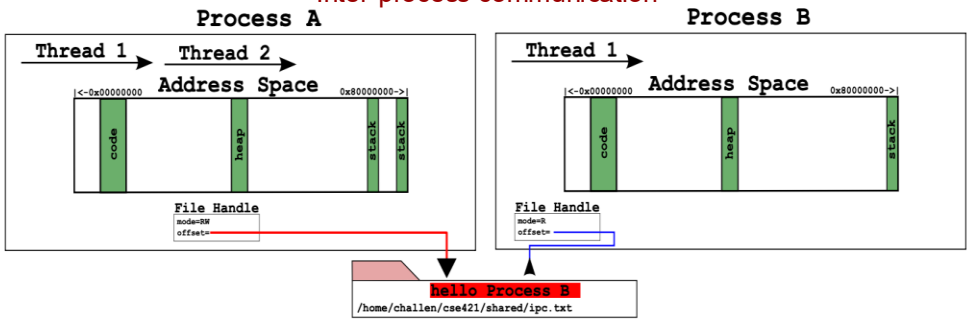
- Thread
  - Execution sequence + CPU state (registers + stack)
  - TCB contains info about a single thread, mostly processor state
  - Communication between threads is cheap and easy (no OS help is needed)
- Process
  - N threads + resources shared by them (hip, global vars, file descriptors, etc.)
  - PCB contains info about containing process, including address space
  - Communication between processes is costly




Intra-process communication

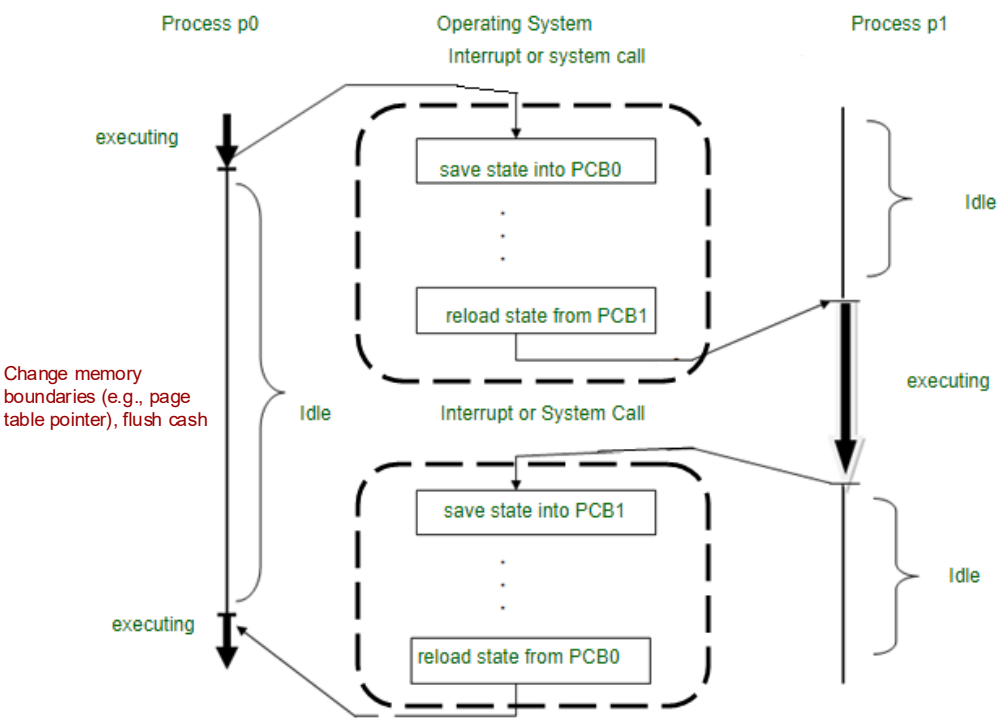


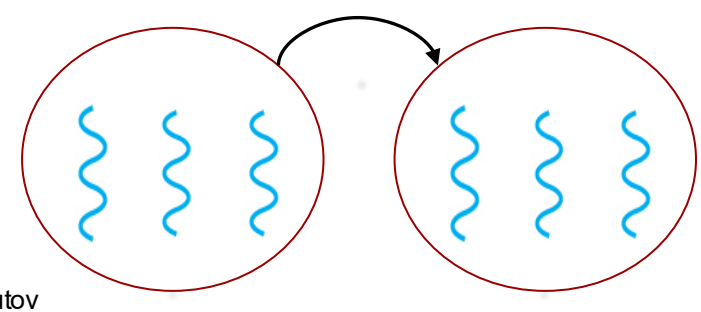

Inter-process communication

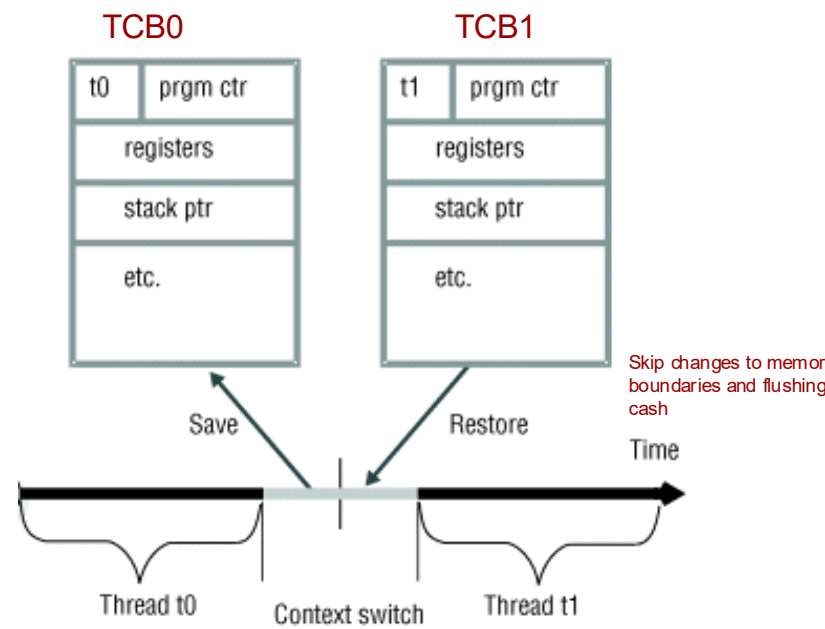


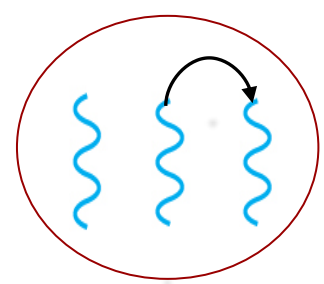

©Tatyana Ryutov

# Review: Process vs. Thread Switch





©Tatyana Ryutov

USC Viterbi School of Engineering

# Example

- Run this program concurrently with multiple threads and processes
- Context switching time: **0.072s** for threads, and **0.784s** for processes

```go
package main

import (
 "fmt"
)

const (
 LIMIT = 30000
)

func exhaust_cpu(limit int) {
 primes := 1
 for i := 3; i <= limit; i += 1 {
  divisible := 1
  for j := 2; j <= limit; j++ {
   if i%j == 0 {
    divisible += 1
   }
  }
  if divisible == 2 {
   primes += 1
  }
 }
 fmt.Println("Primes: ", primes)
}

func main() {
 exhaust_cpu(LIMIT)
}
```

program

```
$ cat threads.go
package main

import (
  "fmt"
  "runtime"
  "sync"
)

const (
  LIMIT = 30000
  THREADS = 5
)

func exhaust_cpu(wg *sync.WaitGroup, limit int){
  defer wg.Done()
  primes := 1
  for i:=3;i<=limit;i+=1 {
    divisible := 1
    for j:=2;j<=limit;j++ {
      if i%j == 0 {
        divisible += 1
      }
    }
    if divisible == 2 {
     primes += 1
    }
  }
  fmt.Println("Primes: ", primes)
}

func main() {
  runtime.GOMAXPROCS(THREADS)
  wg := sync.WaitGroup{}
  for i:=0;i<THREADS;i+=1 {
    go exhaust_cpu(&wg, LIMIT)
    wg.Add(1)
  }
  wg.Wait()
}
```

the program in 1 process with 5 threads

```
# set the threads count to 1 for individual processes
$ diff threads.go procs.go
<   THREADS = 5
---
>   THREADS = 1

$ cat run_procs.sh
PROCS_COUNT=${1:-5}
go build procs.go
for i in $(seq 1 ${PROCS_COUNT})
do
  # Run process in background
  ./procs &
done
# Wait for background processes to complete
wait
echo "Waited for all procs"
```

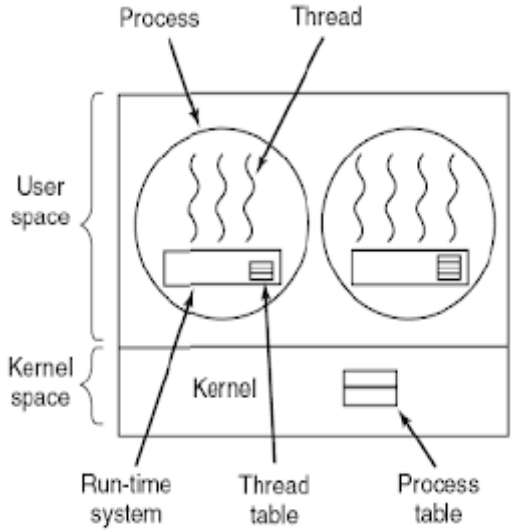
the program in 5 processes, each with 1 thread

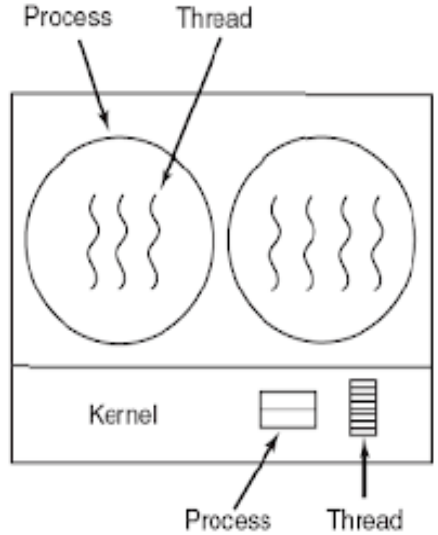
©Tatyana Ryutov



USC Viterbi School of Engineering

# Review: Kernel- and User-level Threads




**User-level threads**

All **code** and **data structures** for the library exist in user space.

Invoking a function in the API results in a **local function call** in user **space** and not a system call.

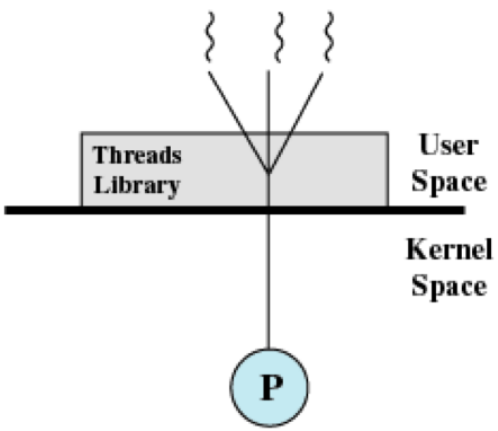
user mode

**Kernel-level threads**

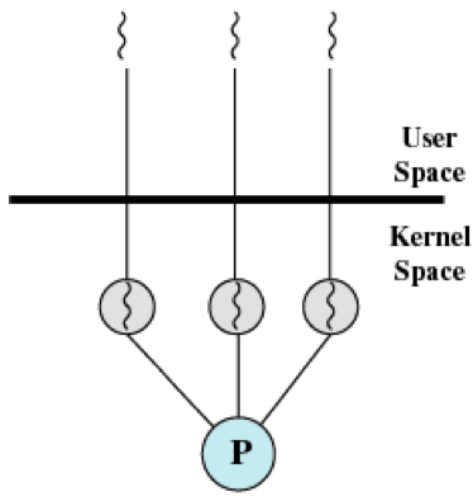
All **code** and **data structures** for the library exists in **kernel space**.

Invoking a function in the API typically results in a **system call** to the kernel.

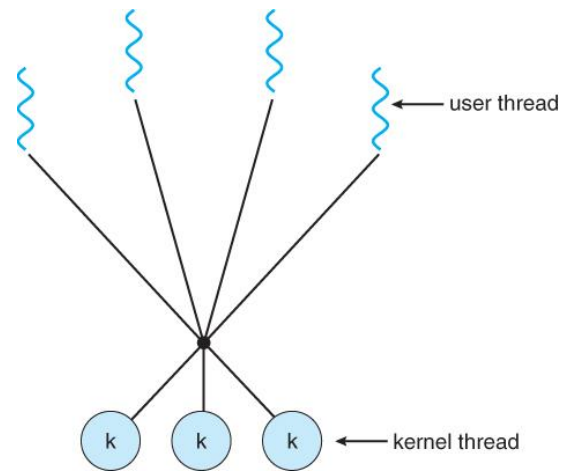
kernel mode




M:1 threading model




1:1 threading model




M:N (M > N) threading model

Student's question: why user-level threads can't multiprocess?

©Tatyana Ryutov

# Outline

- Review
- Synchronization inro
- Synchronization constructs
  - Locks, lock implementation
  - Condition variables
  - Semaphores

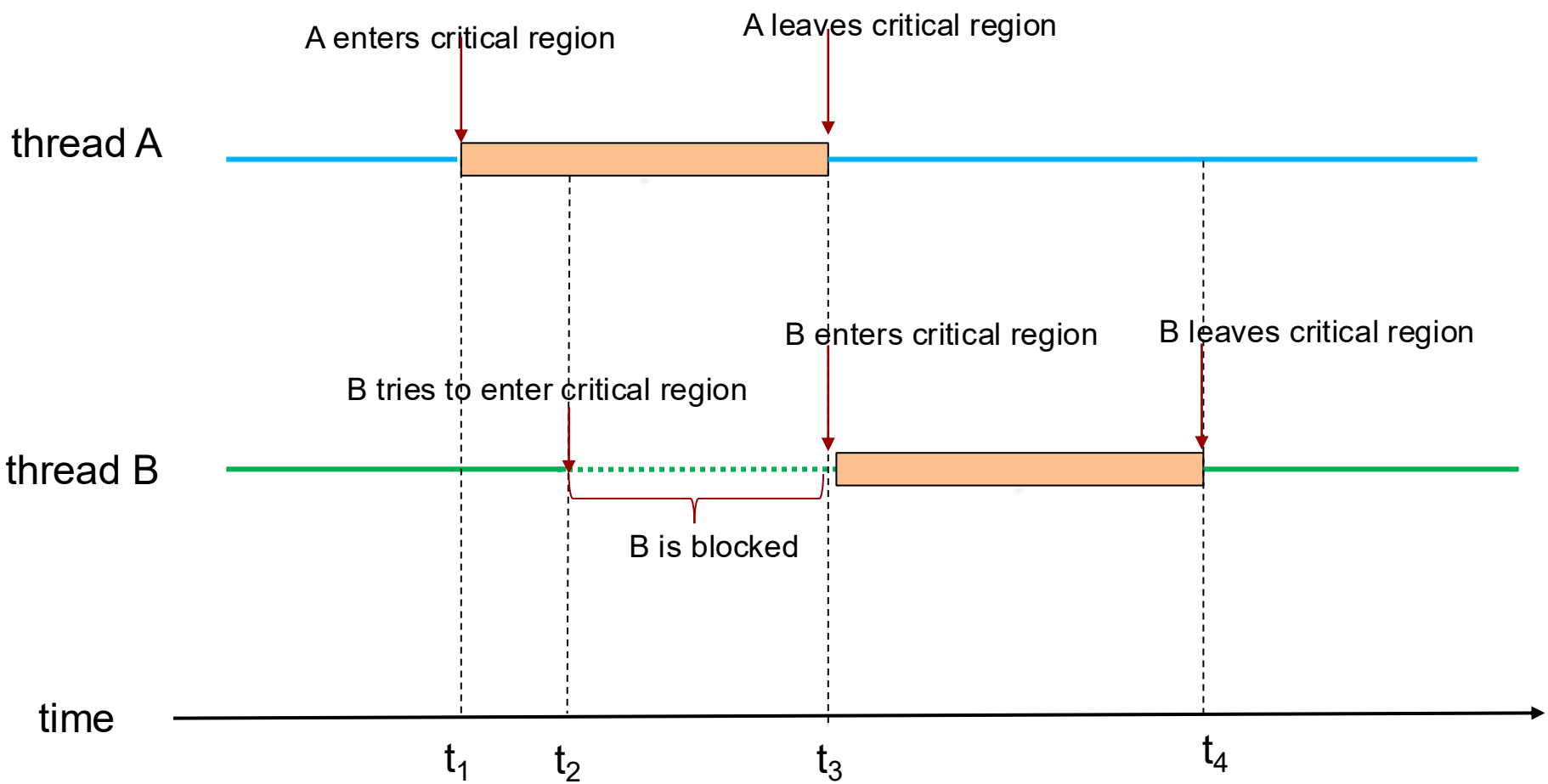
©Tatyana Ryutov

# Definitions

- Synchronization: use atomic (all or nothing) operations to ensure cooperation among threads and avoidance of race conditions
- Mutual exclusion: shared resource can only be accessed by one thread at a time
- Critical section (critical region):
  - Code sequence that is vulnerable to races
  - Every race involves two or more threads executing related critical sections concurrently
- To avoid race conditions ensure **mutual exclusion in critical sections**

©Tatyana Ryutov

# Mutual Exclusion using Critical Regions




©Tatyana Ryutov

USC Viterbi School of Engineering

# General Structure for Critical Sections

- Every thread has the same pattern:
  - **Entry section**: code to attempt entry to critical section, request "permission"
  - **Critical section**: code that requires isolation (e.g., with mutual exclusion)
  - **Exit section**: cleanup code after execution of critical region
  - **Non-critical section**: everything else

```
...
entry section
critical section
exit section
non-critical section
...
```

- There can be multiple critical sections in a program
  - Only critical regions that access the same shared resource (e.g., data structure) need to synchronize with each other

©Tatyana Ryutov

# Solving Critical-Section Problem

- A correct solution to the critical section problem must satisfy:
  - Safety:
    - Nothing bad ever happens!
    - At most one thread may be executing in the critical section (mutual exclusion)
  - Liveness:
    - Eventually something good happens!
    - If one or more threads are in their entry section, then eventually at least one of them enters the critical section
  - Further requirement (making liveness practical):
    - Bounded waiting:
      - No thread should wait forever to enter a critical section



©Tatyana Ryutov

# Outline

- Review
- Synchronization inro
- Synchronization constructs
  - Locks, lock implementation
  - Condition variables
  - Semaphores

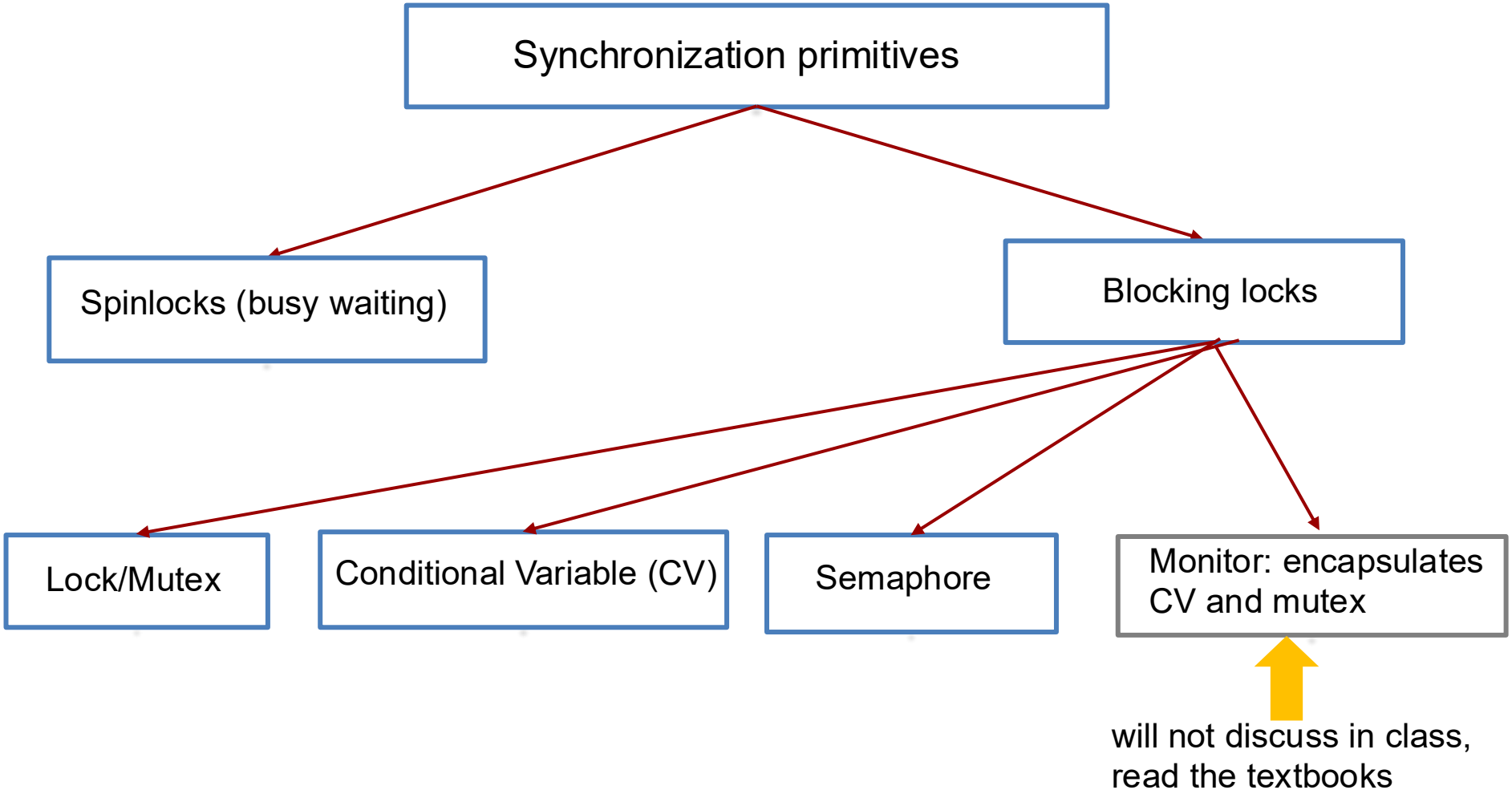
©Tatyana Ryutov

# The Trouble with Threads

- Basic problem
  - Threads interleave executions arbitrarily
  - Scheduling is not under program control
  - If two concurrent threads are accessing a shared variable, and that variable is read/modified/written by those threads, then access to the variable must be controlled to avoid erroneous behavior
- Cooperation is controlled using synchronization
  - Restrict the possible interleavings
- We will look at
  - Mechanisms to control access to shared resources
    - Locks (mutexes), condition variables, semaphores
  - Pattern for coordinating accesses to shared resources
    - Bounded buffer/producer-consumer
  - We'll discuss in terms of threads, also applies to processes

©Tatyana Ryutov

# Synchronization Primitives Overview




©Tatyana Ryutov

# Mutex: Mutual Exclusion

- Mutex is a synchronization variable that provides mutual exclusion
  - Ensures that only one thread can access the shared state at a time
  - Avoids race condition
- Thread should do the following:
  - Lock shared state before entering a critical section
  - Unlock shared state when leaving the critical section
  - Wait if the critical section is locked
- When one thread holds a mutex no other thread can hold it

©Tatyana Ryutov

# Mutex

- A mutex can be in one of two states: BUSY or FREE
  - Initially FREE
- Mutex has a queue of zero or more threads waiting for the lock state to become FREE
- Mutex operations:
  - **`lock.acquire()`**
    - Wait until lock is FREE, then take it and set it to BUSY
      - Checking if the state is FREE and setting it to BUSY is an atomic operation
      - Even if multiple threads try to acquire the lock, at most one thread will succeed, others will be queued
  - **`lock.release()`**
    - Set it to FREE, waking up one waiting thread

©Tatyana Ryutov

# Lock: Properties

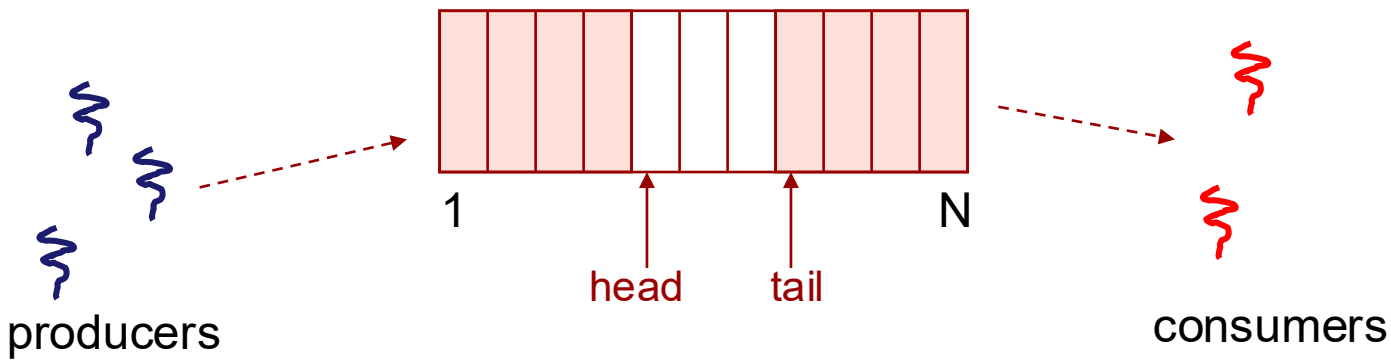
1. At most one lock holder at a time (**safety**)
   – If no one holding, **lock.acquire()** gets the lock (**liveness**)
2. If thread T attempts to acquire a lock, then there exists a bound on the number of times other threads can successfully acquire the lock before T does (**liveness: bounded waiting**)

- NOTE: the total order of accesses to the lock is not guaranteed!

©Tatyana Ryutov

# Access Pattern: Bounded Buffer (Bounded Queue)

- AKA "producer/consumer" problem
  - There is a buffer in memory with N entries
  - Producer threads insert entries into it (one at a time)
  - Consumer threads remove entries from it (one at a time)
  - Buffer is a shared resource
    - Code that manipulates the buffer is a critical section
- Thread-safe bounded buffer is a type of a buffer that is safe to access from multiple concurrent threads
  - We must use synchronization constructs to control access to shared variables describing the buffer state




©Tatyana Ryutov

# Example: Bounded Buffer using Locks

**Consumer**

```
tryget() {
lock.acquire();
   if (head < tail) {
// not empty
    item = buf[head % MAX];
        head++;
    }
    lock.release();
    return item;
}
```

**Producer**

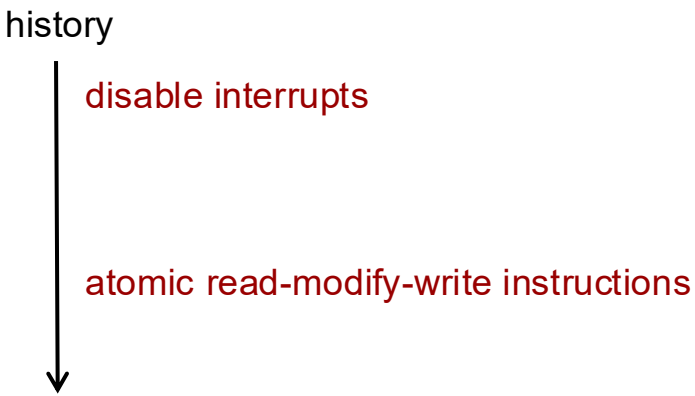
```
tryput(item) {
  lock.acquire();
  if ((tail - head) < MAX) {
// not full
     buf[tail % MAX] = item;
        tail++;
    }
    lock.release();
}
```

- **`head`** is total number of items that have ever been removed
- **`tail`** is total the number of items ever inserted
- Buffer access: add to **`tail`** remove from **`head`**
  - Both operations are modulo **`MAX`**
- Initially: **`head`** = **`tail`**= 0; **`lock`** = **`FREE`**; **`MAX`** is the buffer capacity

©Tatyana Ryutov

# Lock Implementation: HW Support

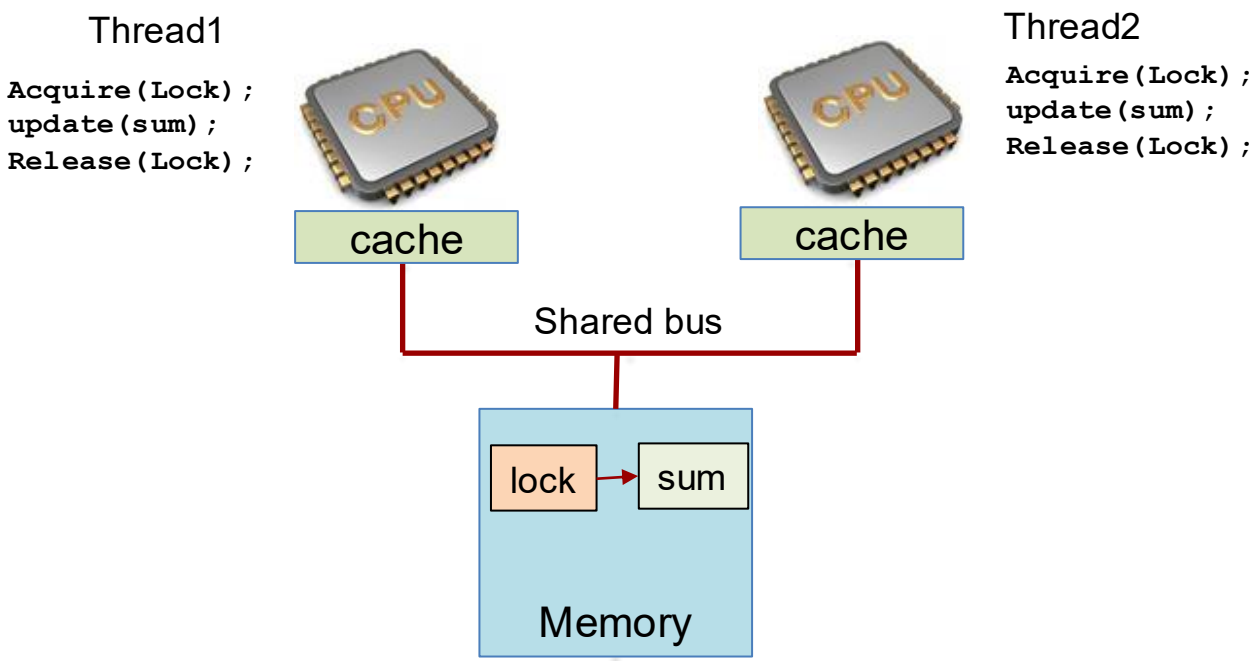
- Lock implementation requires some level of hardware support
- Two hardware primitives:
  1. **Disabling interrupts**
     - On a uniprocessor, we can make a sequence of instructions atomic by disabling interrupts
  2. **Read-Modify-Write instructions (RMW)** support an atomic action:
     1. Read a memory location into a register
     2. Modify the value
     3. Write a new value into the memory location
     - **Globally atomic** with respect to the instructions on every processor




©Tatyana Ryutov

# Why do we need atomic operations?

- Consider lock implementation without support for atomicity:
  - `Acquire(Lock): if (Lock.value == 0) Lock.value = 1`
  - Thread1:
    - T1: reads lock, it's free, context switch, T1': sets lock to busy, reads and updates sum
  - Thread2:
    - T2: reads lock, it's free, sets to busy, reads sum, context switch, T2': updates the sum
  - Update done by thread1 is lost
- HW has to support special instructions that implement atomic operations




©Tatyana Ryutov

# Uniprocessor: Lock Implementation

- **Idea:** impose mutual exclusion only on the operations of testing and setting the shared lock variable
- Disable interrupts and do not call library functions that can trigger a context switch during execution of critical section
  - Enable/disable interrupts is a memory barrier operation – so it forces all memory writes to complete first
- Provides mutual exclusion, but does not work as a general solution
  - Disabling interrupts for long time is costly
  - Does not work for user-level threads

©Tatyana Ryutov

# Uniprocessor: Lock Implementation Example

```
Acquire(Lock) {
    disableInterrupts();
    if (Lock.value != 0) { //lock is BUSY
    {
        add me to the queue of threads waiting for the lock;
        yield(); //switch to next ready thread
    } else {
        Lock.value = 1;
      }
      enableInterrupts();
   }
```

```
Release(Lock) {
    disableInterrupts();
    if (anyone in the queue of threads waiting for the lock) {
      take one waiting thread off the queue, put it on ready queue;
    }
  Lock.value = 0; // lock is FREE
  enableInterrupts();
}
```

©Tatyana Ryutov

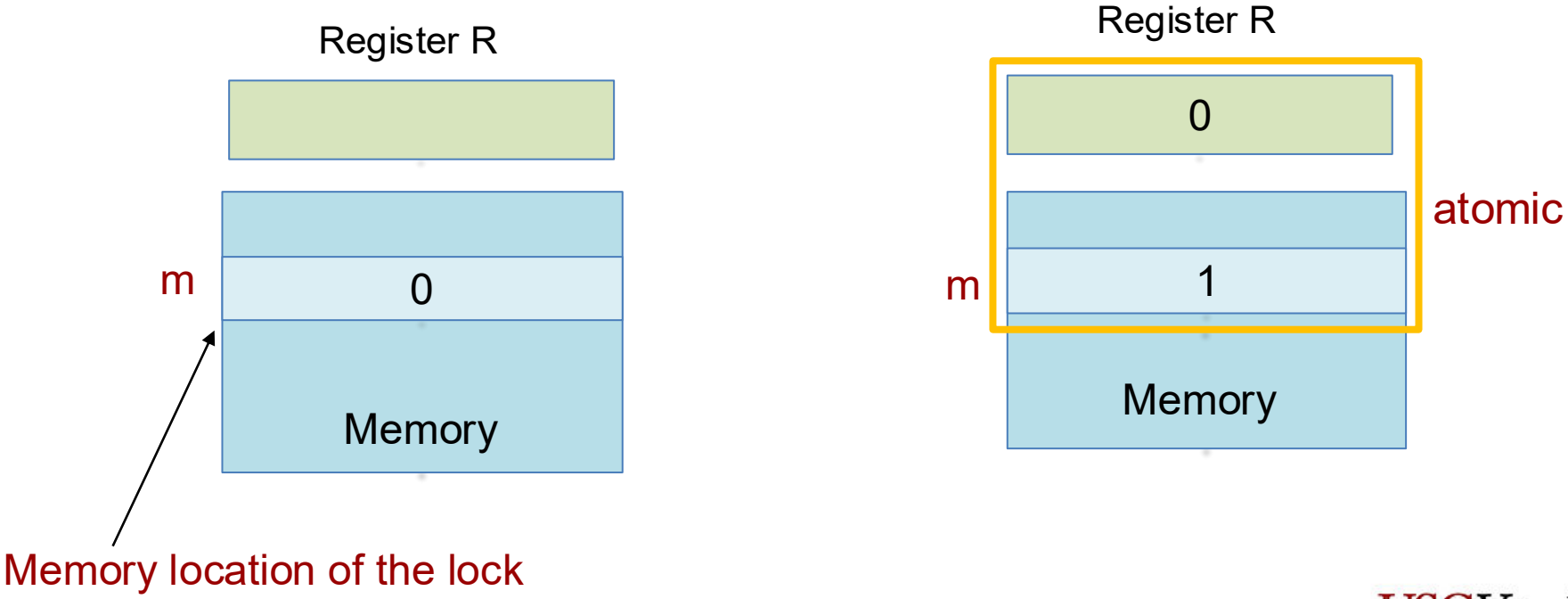
# Multiprocessor: Lock Implementation

- On multiprocessors we need HW support to implement locks
- Read-modify-write (RMW) instructions
  - Perform multi-step operations atomically
    - Atomically read a value from memory, operate on it, and then write back to memory; atomic with respect to all instructions on all other processors
  - Mutual exclusion
    - Intervening instructions prevented in HW
    - Queue all concurrent instructions but one
- Examples:
  - test-and-set (most architectures) – read value, write “1” back to memory
  - exchange (x86) – swaps value between register and memory
  - compare-and-swap read value, if value matches register, do exchange

©Tatyana Ryutov

# Test-and-Set Instruction

- **`test-and-set(R,m)`** writes a new value to a memory location and returns its old value as a single atomic (i.e., non-interruptible) operation
- If multiple threads access the same memory location, and if a thread is currently performing **`test-and-set`**, no other thread may begin another **`test-and-set`** until the first thread's **`test-and-set`** is finished




0: lock is FREE; 1: lock is BUSY

©Tatyana Ryutov

# Critical Section Using Spinlocks

- Spinlock is a lock where the processor waits in a loop for the lock to become free
- Spinlock can be implemented entirely in user space: from OS point of view, a "spinning" thread looks exactly like a thread that does a heavy computation
- Critical section with spinlocks:

```
while (busy) {
acquire lock
}
critical section
release lock
remainder statements
```

busy waiting

Non-blocking lock, so no queue of waiting threads!

©Tatyana Ryutov

# Spinlock using Test-and-Set

- Semantics of **`test-and-set`** operation: spinlock remains unavailable until this operation called on the respective spinlock returns zero
- Memory barrier is required to make sure other threads do not see upcoming changes before the shared data is unlocked
  - All the pending memory operations complete before releasing the lock

```
Acquire(Spinlock) {
   while (test-and-set(Spinlock.value) != 0) //while busy
      ; //spin
}
```

```
Release(Spinlock) {
   memory_barrier();
   Spinlock.value = 0;
}
```

0: lock is FREE; 1: lock is BUSY

©Tatyana Ryutov

# Spinlock Tradeoffs

- Advantage: no context switch is required
  - Assume lock will be held for a short time
- Disadvantage: requires busy waiting
  - Consumes CPU resources
  - Can delay the scheduling of other threads
  - When held for longer time periods, harms both parallelism and concurrency

©Tatyana Ryutov

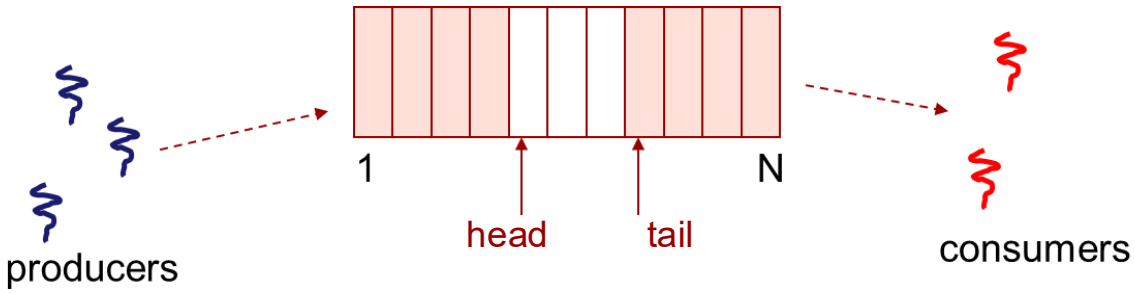
# Outline

- Review
- Synchronization inro
- Synchronization constructs
  - Locks, lock implementation
  - Condition variables
  - Semaphores

©Tatyana Ryutov

# Beyond Locks

- Recall the Bounded Buffer problem: producer puts things in a fixed sized buffer, consumer takes them out
- What are the constraints for the bounded buffer?
- Solution: condition variables
  - An abstraction that supports conditional synchronization
  - Condition variable is always associated with a lock
    - Enables threads to wait inside a critical section by releasing the lock




©Tatyana Ryutov

# Condition Variable (CV)

- Condition variable (CV) is a synchronization primitive that
  - Communicates info about tstate of shared data
  - Allows explicit event notification
  - Blocks thread until some desired event occurs
- Without condition variable threads continually have to poll to check if the condition is met
- CV provides signaling (notification mechanism), not mutual exclusion
  - A mutex is needed to synchronize access to the shared data
- CV operations:
  - **`wait()`** atomically unlocks the mutex and blocks the thread
  - **`signal()`** unblocks **one** of the threads waiting on CV
  - **`broadcast()`** unblocks **all** of the threads waiting on CV
- CV has no internal state other than a queue of waiting threads

**Consumer:**

```
while(buffer.empty())
{
    sleep(1);
}
```

©Tatyana Ryutov

# CV Traps

- Lost wakeup: causes a thread to miss a wake-up due to race condition
  - Sender sends its notification before the receiver gets to its wait state
  - The consequence is that the notification is lost
- Spurious wakeup: thread wakes up, although the condition it is waiting for is not satisfied
  - Result of a race condition between threads waiting on CV
  - When lock is released, anyone can acquire it
    - Even if the desired predicate was true when signal or broadcast was called, it may no longer be true when wait returns
  - For safety, **`wait`** must be called from within a loop to test the predicate:

```
while (needToWait()) {
    wait(cond, mutex);
}
```

©Tatyana Ryutov

# CV: Usage Pattern

guards against spurious wakeup (but does not prevent it)

guards against lost wakeup

```
lock(&mutex);                    /* lock mutex to safely examine the predicate,
                                    by preventing other threads from altering it */
while (predicate == FALSE) {     /* check predicate */
  wait(&cond, &mutex);           /* atomically go to sleep and release mutex,
                                    recheck predicate when awoken */
}
do_stuff();                      /* do whatever you need to do when predicate
                                    becomes true */
unlock(&mutex);            /* unlock mutex */
```

```
lock(&mutex);     /* lock the mutex to ensure exclusive access
                     to the shared state that involves the predicate */

predicate = TRUE;        /* set the predicate   */
signal(&cond);            /* wakeup one of the threads waiting on the
                             predicate (if any), waking thread goes on the queue
                             waiting for the lock */
unlock(&mutex);   /* unlock the mutex to allow others to proceed */
```

©Ta

# CV: Typical Scenario

**Main Thread**

- Declare and initialize data/variables which require synchronization (e.g., "count")
- Declare and initialize a condition variable object
- Declare and initialize an associated mutex
- Create threads A and B to do work

**Thread A**

- Do work up to the point where a certain condition must occur (such as "count" must reach a specified value)
- Lock associated mutex and check the value of a shared variable
- Call `wait()` to perform a blocking wait for signal from Thread B
- Call `wait()` automatically and atomically unlock the associated mutex variable so that it can be used by Thread B
- When signaled, wake up. Mutex is automatically and atomically locked
- Explicitly unlock mutex
- Continue

**Thread B**

- Do work
- Lock associated mutex
- Change the value of the variable that Thread A is waiting upon
- Check value of the shared variable, if it fulfills the desired condition, signal Thread A
- Unlock mutex
- Continue

**Main Thread**

- Join/Continue

# CV: Example

The main routine creates four threads
Two of the threads perform work and update **count** variable
The other two wait until **count** reaches a specified value

```
#define NUM_THREADS 4
#define TCOUNT 1000
#define COUNT_LIMIT 100

int count = 0;
int thread_ids[4] = {0,1,2,3};

mutex_t count_mutex;
cond_t count_threshold_cv;

int main (int argc, char *argv[]){
long t1=1, t2=2, t3=3, t4=4;

thread_t threads[4];

thread_create(&threads[0],  watch_count, (void *)t1);
thread_create(&threads[1],  watch_count, (void *)t2);
thread_create(&threads[2],  inc_count, (void *)t3);
thread_create(&threads[3],  inc_count, (void *)t4);

for (i=0; i < NUM_THREADS; i++) {  /* Wait for all threads */
thread_join(threads[i]);
}
```

©Tatyana Ryutov

# CV: Example Contd.

```
void *inc_count(void *t){
long my_id = (long)t;
for (int i=0; i < TCOUNT; i++) {
  lock(&count_mutex);
  count++;
  if (count == COUNT_LIMIT) {
    cond_signal(&count_threshold_cv);
    }
    unlock(&count_mutex);
   }
    thread_exit(NULL);
  }

void *watch_count(void *t){
long my_id = (long)t;
lock(&count_mutex);

while (count < COUNT_LIMIT) {
  cond_wait(&count_threshold_cv, &count_mutex);

 }
  count -= 100;
  unlock(&count_mutex);
  thread_exit(NULL);
}
```

lock is re-acquired again before returning from **wait**
There is no guarantee that the desired condition still holds: another thread might have snuck in
So, need to check the condition again

lock acquired

lock released

lock acquired

©Tatyana Ryutov

# Bounded Buffer using CV

## Consumer

```
get() {
    lock.acquire();
    while (head == tail)      {
  // buffer empty
        empty.wait(lock);
   }
    item = buf[head % MAX];
    head++;
    full.signal(lock);
    lock.release();
    return item;
}
```

## Producer

```
put(item) {
 lock.acquire();
 while ((tail - head) == MAX) {
   // buffer full
        full.wait(lock);
    }
     buf[tail % MAX] = item;
     tail++;
     empty.signal(lock);
     lock.release();
}
```

- **`head`** is total number of items that have ever been removed
- **`tail`** is total the number of items ever inserted



©Tatyana Ryutov

USC Viterbi School of Engineering

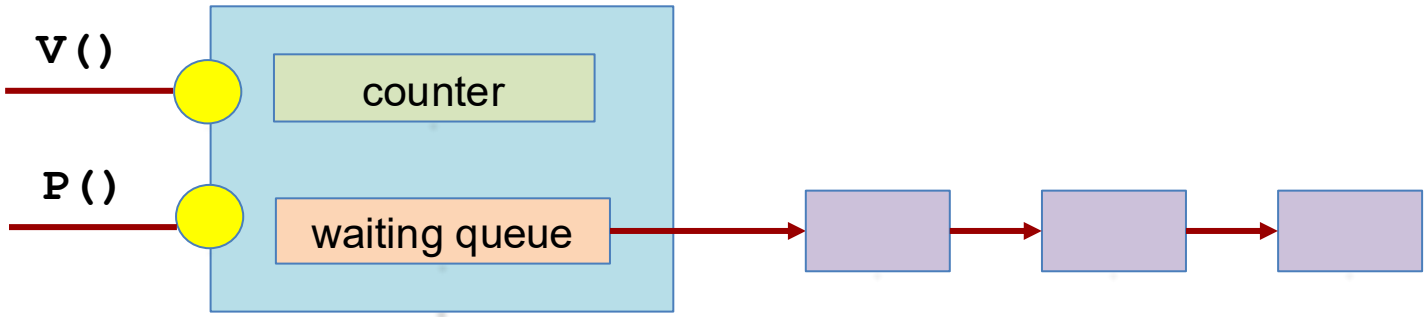
# Outline

- Review
- Synchronization inro
- Synchronization constructs
  - Locks, lock implementation
  - Condition variables
  - Semaphores

©Tatyana Ryutov

# Semaphore

- Semaphore - non-negative integer variable (counter) with two atomic operations and a waiting queue
- When a semaphore is initialized, the counter is set to a specified value
- Operations:
  - **`P()/down()/wait()`**
    - Try to decrement the counter by 1
    - If the counter is zero, the thread will block until it is positive, and when it is unblocked it will decrement the counter
  - **`V()/up()/signal()`**
    - Increment the counter by 1, wake up one thread waiting in **`P()`**




©Tatyana Ryutov

# Semaphore Contd.

- Critical section with semaphores:
  - get access to semaphore with **`P()`**
  - perform critical region operations
  - release semaphore with **`V()`**
  - remainder statements
- Semaphore has state (while a CV is memoryless)
  - So **`V()`** in the absence of waiters allows next **`P()`** to immediately proceed
- The counter keeps track of the number of threads that do not have to block
- If initial semaphore value is 1, then semaphore acts as a mutex

©Tatyana Ryutov

# Bounded Buffer using Semaphore

- Semaphores:
  1. **`mutex`** protects the shared buffer **`buf;`** initialized to 1 (free)
  2. **`empty`** counts empty slots, if **`MAX`** nothing to get; initialized to **`MAX`**
  3. **`full`** counts full slots, if **`MAX`** nowhere to put; initialized to 0

Consumer

```
get() {
    full.P(); /* Decrement number
                 of full slots */
    mutex.P();  // lock
    item = buf[head % MAX];
    head++;
    mutex.V(); // unlock
    empty.V(); /* Increment number
                  of empty slots */
    return item;
}
```

Producer

```
put(item) {
    empty.P(); /* Decrement number
                  of empty slots */
    mutex.P();  // lock
    buf[tail % MAX] = item;
    tail++;
    mutex.V(); // unlock
    full.V(); /* Increment number
                 of full slots */
}
```

©Tatyana Ryutov

# Semaphore vs. Mutex

- Locks: mutual exclusion only (1-exclusion)
  - Mutex is taken and released, always in that order, by each thread that uses the shared resource it protects
- Semaphores: N-exclusion (N – initial counter value)
  - It allows a resource to be concurrently accessed by at most N threads at any time; N is application-dependent

©Tatyana Ryutov

# Lock/CV/Semaphore

- Mutex = flag + waiting queue
  - Use mutex where you just want to allow a piece of code (critical section) to be executed by one thread at a time
- CV = waiting queue + signaling mechanism
  - Use it to avoid busy waiting for some condition to become true, requires mutex
- Semaphore = counter + waiting queue + signaling mechanism
  - Use it to coordinate access to shared resources
  - Semaphore can be used as a mutex if it's counter value is set to 1 (binary semaphore)
    - Differences with mutex?

©Tatyana Ryutov

# Remember the Rules

- Always use locks and condition variables
  - The textbook (T1) focuses on protecting shared objects using locks and CVs for synchronization, not semaphores because semaphores are error prone
- Always acquire lock at beginning of procedure, release at end
  - Never acquire a lock in a parent thread and pass it to a child
- Always hold lock when using a condition variable
  - Always wait in while loop
- Never call sleep() to have one thread wait for another thread to perform a task (synchronize)
  - Wait on a mutex or condition variable instead

©Tatyana Ryutov

# Comparison of Synchronization Primitives

| | Blocking | Signaling | Mutual Exclusion | Ownership |
|---|---|---|---|---|
| Mutex | yes | no | yes | yes |
| Spinlock | no | no | yes | yes |
| Conditional Variable | yes | yes | no | no |
| Semaphore | yes | yes | yes, if set to1 | no |

©Tatyana Ryutov